Habeshan REMS

Software Requirements Specification

Remote Employee Management System for Ethiopian Outsourcing Agencies

Version 1.1 | August 12, 2026 Project: Habeshan REMS | Group 5 | INSA Cyber Talent Program
Conformance: IEEE Std 830-1998 - Recommended Practice for Software Requirements Specifications

---

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the complete functional and non-functional requirements of Habeshan REMS (Remote Employee Management System), a web-based platform that enables Ethiopian outsourcing agencies, software development companies, and other distributed teams to manage remote employees with confidence. The document is written in conformance with IEEE Std 830-1998 and is intended to serve as the single source of truth for design, implementation, testing, and acceptance of the Release 1.0 (MVP) system.

The intended audience for this document includes: the Habeshan REMS development team (frontend, backend, and database engineers), the course instructor and academic evaluators, and prospective pilot customers or investors evaluating the technical maturity of the product. Readers are assumed to have a basic understanding of web application architecture; no prior knowledge of the Habeshan REMS codebase is required.

1.2 Scope

Habeshan REMS is a centralized, offline-first platform for managing distributed employees and their daily work activities. It replaces the spreadsheets, chat threads, and informal check-ins that Ethiopian outsourcing agencies currently rely on, giving managers verifiable visibility into attendance, task progress, and hours worked — the confidence they need to let employees work remotely instead of commuting to a physical office.

1.2.1 In-Scope (Release 1.0 / MVP)

- Account authentication, password reset, and role-based access control for three roles: Admin, Manager, Employee.
- Multi-organization support: each subscribing agency operates within an isolated organization workspace.
- Offline-capable employee clock-in / clock-out with tamper-evident local queuing and automatic synchronization.
- Task creation, multi-employee assignment, Kanban-style status tracking, and an online per-task work timer with pause/resume and reason capture. Offline task-timer queuing and synchronization are deferred to Release 1.1 (see FR-TASK-10, Section 3.1.3).

- Fully automated timesheet generation from attendance and task-timer data, with employee review/submission and manager approval or rejection.
- Manager dashboard summarizing team attendance, task progress, and pending approvals.
- A lightweight, opt-out performance leaderboard to encourage engagement.
- Telegram Bot notifications for reminders, approvals, and rejections.
- Attendance and task reporting with CSV / Excel / PDF export.
- ETB-denominated organizational and billing display (seat count, plan tier) — full payment processing is Release 1.1, see Section 2.6.

1.2.2 Out-of-Scope (Release 1.0 / MVP)

The following are explicitly excluded from Release 1.0 to protect the delivery timeline; each is formally deferred in Section 2.6 (Apportioning of Requirements):

- Automated payroll calculation and bank / mobile-money disbursement.
- In-app payment processing via Chapa or Telebirr (deferred to Release 1.1).
- Random desktop screenshot capture or continuous activity surveillance.
- AI-based productivity or sentiment analysis.
- Git / IDE integration for automatic proof-of-work capture.
- Native iOS / Android applications (the MVP ships as an installable PWA only).
- Real-time chat within tasks (Telegram is the supported channel for the MVP).
- Email or SMS notification channels.
- Sub-manager / multi-level management hierarchies beyond Admin–Manager–Employee.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
SRS	Software Requirements Specification
REMS	Remote Employee Management System
MVP	Minimum Viable Product — the first releasable feature set
PWA	Progressive Web App — a browser-based application that can be installed and used offline via a Service Worker
Admin	User responsible for organization-wide configuration, user accounts, and role assignment
Manager	User responsible for supervising a team: assigning tasks, monitoring attendance, approving timesheets
Employee	User who performs assigned work, records attendance, and submits timesheets
Attendance	Record of an employee's clock-in and clock-out activity for a work session
Timesheet	System-generated record of hours worked, derived from attendance and task-timer logs

Term	Definition
Kanban Board	Visual board organizing tasks into columns by status (To Do / In Progress / Completed)
Offline Queue	Local, encrypted browser storage (IndexedDB via Dexie.js) holding actions recorded without connectivity
Synchronization	Process of transmitting queued offline records to the server once connectivity is restored
RBAC	Role-Based Access Control — restricting system functionality according to a user's assigned role
ETB	Ethiopian Birr — the system's default currency for organizational and billing display
API	Application Programming Interface
JWT	JSON Web Token — a signed, stateless token used to authenticate API requests
Telegram Bot	An automated Telegram account, integrated via the Telegram Bot API, used to push reminders and status alerts
Proof-of-Work	Evidence (task-timer logs, status updates, timestamps) demonstrating that assigned work was performed
Device Hash	A cryptographic fingerprint computed on-device at the moment of a clock event, used to detect tampering with a queued offline record; its construction and threat model are defined in Section 3.1.2
Seat	One billable employee account within an organization's subscription plan
Fiber	A Go web framework, used for the Habeshan REMS backend, modeled on Express.js routing conventions
GORM	The object-relational mapping library used to interact with PostgreSQL from the Go backend
Dexie.js	A JavaScript wrapper library that simplifies use of the browser's IndexedDB for offline data storage
Tamper-Evident	A design property ensuring that unauthorized modification of locally stored data is detectable by the server during synchronization, within the threat model defined in Section 3.1.2
UUID	Universally Unique Identifier — a 128-bit value used as the primary key type for every table in the schema (default `gen_random_uuid()`)
timestamptz	PostgreSQL's timezone-aware timestamp type, used for every timestamp column in the schema
BaseModel	The shared Go struct (`backend/internal/common/base.go`) that every GORM model embeds; establishes each table's `id`, `org_id` (where applicable), `created_at`, and

Term	Definition
	`updated_at` columns, and `deleted_at` where soft delete applies

1.4 References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
- IEEE/EIA 12207.1-1997, Guidelines for Software Life Cycle Data (referenced via IEEE Std 830-1998, Annex B).
- Telegram Bot API official documentation (core.telegram.org/bots/api).
- Go Fiber web framework documentation (docs.gofiber.io).
- GORM object-relational mapper documentation (gorm.io).
- Dexie.js documentation for IndexedDB access (dexie.org).
- PostgreSQL 15 official documentation (postgresql.org/docs).
- Chapa and Telebirr merchant API references, for planned Release 1.1 billing integration.
- `contracts/db-schema.md`, Habeshan REMS backend repository — the database contract this specification conforms to.

1.5 Document Overview

The remainder of this document is organized as follows:

- **Section 2 — Overall Description:** product perspective, architecture, user classes, operating environment, design constraints, apportioning of requirements, and assumptions.
- **Section 3 — Specific Requirements:** the complete, testable set of functional requirements, external interface requirements, non-functional requirements, database schema, state machines, and business rules.
- **Section 4 — System Models:** use case, entity-relationship, and state-machine diagrams that visualize the logic described in Section 3.
- **Section 5 — Appendices:** use case catalog, full REST API surface, requirements traceability matrix, environment configuration template, and future-release roadmap.

---

2. Overall Description

2.1 Product Perspective and Operating Architecture

Habeshan REMS is a new, standalone, three-tier web application. It is not a module or extension of an existing system. Its defining architectural characteristic is offline-first operation on the client tier: because connectivity in Addis Ababa and other Ethiopian markets is frequently interrupted, an employee's clock-in and clock-out actions must always succeed locally, even without a network connection, and reconcile with the server automatically once connectivity returns.

The system is composed of a Progressive Web App (PWA) client, a stateless Go REST API, a PostgreSQL relational database, and an integrated Telegram Bot notification service.

Layer	Technology	Hosting / Notes
Presentation (Client)	React 18 SPA built with Vite, TypeScript, Tailwind CSS, Shadcn UI components; Dexie.js over IndexedDB for the offline queue; installable as a PWA via a Service Worker.	Vercel or Netlify (static hosting, zero server cost)
Application (Backend)	Go 1.22+, Fiber web framework, GORM ORM, golang-jwt for JSON Web Tokens, gopkg.in/telebot.v3 for the Telegram Bot integration.	Render or Railway
Data	PostgreSQL 15, 11 normalized tables (see Section 3.4).	Aiven Cloud (managed Postgres, free tier for MVP)

2.2 Product Functions

The following is a summary of the major functions Habeshan REMS performs; each is elaborated with testable requirements in Section 3.1.

- Authentication, password recovery, and role-based authorization.
- Offline-capable attendance capture with tamper-evident synchronization.
- Task creation, multi-employee assignment, Kanban status tracking, and a work timer with pause/resume.
- Fully automated timesheet generation, employee review, and manager approval workflow.
- Manager dashboard and lightweight performance leaderboard.
- Telegram notifications for reminders and status changes.
- Attendance and task reporting with export.
- Organization- and role-scoped administration.

2.3 User Classes and Characteristics

Role	Description	Technical Level	Key Actions
Admin	Configures the organization workspace and manages accounts. Typically an agency owner or operations lead.	Basic–Intermediate	Create/deactivate users, assign roles, configure organization settings, view org-wide reports
Manager	Supervises one or more teams; the primary consumer of accountability data.	Basic	Assign tasks, monitor attendance and task progress, approve/reject timesheets, view dashboard and reports
Employee	Performs assigned work remotely; the primary user of the offline attendance and task features.	Basic	Clock in/out, work on tasks with the timer, review and submit auto-generated timesheets, receive notifications

2.4 Operating Environment

Client Environment

- Desktop and laptop computers, and mobile devices, via modern evergreen browsers (Chrome, Firefox, Edge, Safari — latest two major versions).
- Minimum supported viewport width: 320px (mobile-first responsive design).
- Installable as a PWA on supported platforms for an app-like offline experience.

Server Environment

- Linux-based container runtime executing a single compiled Go binary (Fiber framework).
- PostgreSQL 15 managed instance (Aiven Cloud).
- REST/JSON API secured with JWT bearer tokens over HTTPS.

Network Environment The system assumes intermittent, not continuous, internet connectivity. Attendance capture must function fully offline; all other functions (task management, dashboards, timesheet approval) require connectivity and display a clear offline indicator when unavailable.

2.5 Design and Implementation Constraints

The following constraints bind the design of Release 1.0:

ID	Constraint	Rationale
C1	Frontend shall be a client-side-rendered SPA (React + Vite), not a server-rendered framework.	Server-side rendering cannot render the UI while offline; a fully client-side bundle can.
C2	Frontend language is TypeScript, not plain JavaScript.	Shared, compile-time-checked data contracts between the offline queue, the API client, and UI components prevent an entire class of synchronization bugs.
C3	Backend language is Go (Fiber framework), not Node.js.	Goroutine-based concurrency comfortably handles simultaneous offline-batch sync, Telegram webhooks, and scheduled timesheet generation on minimal server resources.
C4	Offline storage uses IndexedDB via Dexie.js; no reliance on localStorage for structured data.	IndexedDB supports larger, structured, queryable offline datasets suitable for attendance and task queues.
C5	All monetary values are denominated in ETB (Ethiopian Birr).	The target market transacts in Birr; USD-denominated tooling is a documented adoption barrier for Ethiopian agencies.
C6	Hosting must fit within free or near-free tiers for the MVP phase.	Startup budget constraint during the pilot phase.

ID	Constraint	Rationale
C7	Telegram is the only supported notification channel for Release 1.0.	Free, already popular in Ethiopia, and has a mature Go client library (telebot.v3); avoids SMTP/SMS cost and setup overhead.
C8	The database schema (Section 3.4) shall use UUID primary keys (default `gen_random_uuid()`) and `timestamptz` timestamp columns. Direct `org_id` foreign keys shall appear only on `users`, `teams`, `attendance_logs`, `tasks`, `task_time_logs`, `timesheets`, and `notifications`. The `organizations` table is the tenant root and does not reference itself. `team_members`, `task_assignments`, and `telegram_subscribers` inherit organization scope through their parent or owning records; this selective placement is an intentional design choice, not an omission.	UUID keys avoid primary-key collisions across a multi-tenant system without a central sequence. Direct `org_id` on tables queried independently makes tenant isolation (NFR-SEC-03) hard to accidentally miss; omitting it on junction/extension tables avoids a redundant second source of truth for an organization boundary already enforced upstream.

2.6 Apportioning of Requirements

In conformance with IEEE Std 830-1998, Clause 5.2.6, this subsection identifies requirements that are explicitly deferred beyond Release 1.0. Section 3 (Specific Requirements) contains only Release 1.0 (MVP) requirements; no deferred feature is described as a testable requirement in this document.

2.6.1 Release 1.1 — Monetization and Deferred Capabilities

Planned after MVP validation. The following capabilities are not Release 1.0 acceptance requirements:

- Per-seat, ETB-denominated subscription billing (Basic / Pro / Enterprise tiers).
- Payment collection via Chapa and Telebirr merchant APIs.
- Self-service plan upgrades/downgrades and seat management for Admin users.
- Invoice history and receipt export.
- Offline task-timer operations and synchronization (FR-TASK-10): local encrypted event queuing, unique `record_uuid` values, integrity validation, automatic replay after reconnection, duplicate handling, and the `pending_sync → synced_verified / rejected_tampered` state flow.

Release 1.0 task-timer behavior: start, pause, resume, and stop require an active network connection. Failed requests shall be clearly reported as not recorded; the interface shall not present an unsuccessful operation as a successfully recorded timer event.

2.6.2 Release 2.0+ — Long-Term Roadmap

- Advanced payroll computation and bank / mobile-money salary disbursement.
- Client portal for international outsourcing clients to view delivery progress.
- Advanced proof-of-work: GitHub/GitLab commit linkage, screenshots, file submission.

- Native mobile applications (iOS/Android) with background offline sync.
- AI-assisted productivity and workload analytics.
- Multi-organization data isolation hardening for large-scale, self-serve growth.

2.7 Assumptions and Dependencies

Assumptions

- Users have access to a modern browser with JavaScript enabled.
- Employees possess a compatible device (desktop, laptop, or smartphone) and will have internet connectivity at least once per work session for synchronization.
- Managers review and act on submitted timesheets in a timely manner.
- Organizations provide accurate employee and team information at onboarding.
- Email verification is not required at registration for the MVP; accounts are provisioned by an Admin or self-registered and then approved.
- Device clocks may be inaccurate or manipulated; the server, not the client, is the source of truth for time once a record is verified (see Section 3.5.1).

Dependencies

- Availability of the Telegram Bot API for outbound notifications.
- Availability of the Aiven-hosted PostgreSQL instance.
- Availability of Render/Railway (backend) and Vercel/Netlify (frontend) hosting platforms.
- Continued browser support for the Service Worker and IndexedDB APIs on target devices.
- For Release 1.1 only: availability and stability of the Chapa and Telebirr merchant APIs.

---

3. Specific Requirements

This section contains every Release 1.0 (MVP) requirement to a level of detail sufficient for design and test-case derivation, in conformance with IEEE Std 830-1998, Clause 5.3. Every requirement is uniquely identified, stated as a single “shall” statement, and tagged MUST (the software is unacceptable without it) or SHOULD (it enhances the product but its absence would not block acceptance) per Clause 4.3.5.2.

3.1 Functional Requirements

3.1.1 Authentication and Access Control (FR-AUTH)

ID	Requirement	Priority
FR-AUTH-01	The system shall allow a user to authenticate using a registered email address and password.	MUST
FR-AUTH-02	The system shall hash and store all passwords using bcrypt with a minimum cost factor of 10; plaintext passwords shall never be stored or logged.	MUST
FR-AUTH-03	The system shall issue a signed JWT (JSON Web Token) upon successful	MUST

ID	Requirement	Priority
	authentication, scoped to the user's organization and role.	
FR-AUTH-04	The system shall validate the JWT on every protected API route and reject missing or invalid tokens with HTTP 401.	MUST
FR-AUTH-05	The system shall enforce role-based access control (RBAC) on every protected endpoint and reject requests from an authenticated but insufficiently privileged role with HTTP 403.	MUST
FR-AUTH-06	The system shall allow an authenticated user to log out, invalidating the client-held token.	MUST
FR-AUTH-07	The system shall allow a user to request a password reset via a time-limited reset token, and to set a new password using a valid token.	MUST
FR-AUTH-08	The registration endpoint shall force the role field to a safe default (Employee) regardless of any client-supplied role value, preventing role escalation at signup.	MUST
FR-AUTH-09	The system shall support soft deletion of user accounts via a `deleted_at` timestamp, without physically removing the row or its historical associations.	SHOULD

3.1.2 Attendance and Offline Synchronization (FR-ATT)

ID	Requirement	Priority
FR-ATT-01	The system shall allow an authenticated Employee to clock in, recording employee ID, organization ID, timestamp, and a client-generated device hash.	MUST
FR-ATT-02	The system shall allow an authenticated Employee to clock out, recording the timestamp and computing session duration; a clock-out shall be rejected with a descriptive error if no corresponding open clock-in exists.	MUST
FR-ATT-03	The system shall allow clock-in and clock-out actions to be recorded while the device has no network connectivity, queuing the action locally in the Dexie.js/IndexedDB offline store.	MUST
FR-ATT-04	Each queued offline attendance record shall carry a client-computed device hash derived from the canonical record contents and a defined local clock reference. The hash is a tamper-detection signal within the documented client-side threat model, not proof against an attacker able to modify both the stored record and its hash.	MUST
FR-ATT-05	When connectivity is restored, the client shall automatically transmit all queued offline records to the server without requiring manual user	MUST

ID	Requirement	Priority
	action.	
FR-ATT-06	The server shall validate each synchronized record against the defined canonical record format, submitted integrity data, timestamp plausibility rules, and available server-side evidence. It shall mark the record `SYNCED_VERIFIED` when validation succeeds or `REJECTED_TAMPERED` when a defined validation check fails. A client-generated hash alone is not independent cryptographic proof of authenticity.	MUST
FR-ATT-07	A record flagged `REJECTED_TAMPERED` shall not count toward verified attendance totals and shall appear in a flagged-review queue visible to the affected Employee and the Employee's authorized Manager. The Manager may review the record and document a review outcome. Any correction or administrative resolution shall be performed only by an authorized Admin, shall retain the original submitted values, and shall record actor, timestamp, and reason. A flagged record shall never be silently converted into a verified record.	MUST
FR-ATT-08	The system shall prevent duplicate synchronization of the same offline record (idempotent sync) using a client-generated record UUID.	MUST
FR-ATT-09	The system shall allow an Employee to view their own attendance history; a Manager shall view attendance only for teams and employees assigned to that Manager; an Admin shall view attendance organization-wide. Supplying a `team_id` or `user_id` query parameter shall not by itself grant access.	MUST

3.1.3 Task Management (FR-TASK)

ID	Requirement	Priority
FR-TASK-01	The system shall allow a Manager (or Admin) to create a task with title, description, priority, due date, and organization ID.	MUST
FR-TASK-02	The system shall allow a task to be assigned to one or more Employees simultaneously (many-to-many assignment).	MUST
FR-TASK-03	An Employee shall see only tasks assigned to them; a Manager shall see all tasks created for their team.	MUST
FR-TASK-04	The system shall support the task-status values To Do, In Progress, Paused, Blocked, and Completed, and shall only allow the transitions defined in Section 3.5.2.	MUST
FR-TASK-05	The system shall provide a per-task work timer that an Employee can start, pause, resume, and stop.	MUST

ID	Requirement	Priority
FR-TASK-06	Pausing an active timer shall require the Employee to select a pause reason (e.g., Power Outage, Personal Break, Commute/Travel, Waiting on Dependency, Other) before the timer stops.	MUST
FR-TASK-07	Task pause reasons shall be visible only to the assigned Employee, their Manager, and Admins; peer employees shall not have access to pause reasons.	MUST
FR-TASK-08	The system shall record every timer start/pause/resume/stop event with a timestamp and duration in the `task_time_logs` table.	MUST
FR-TASK-09	A Manager shall be able to monitor aggregate task progress (counts by status, overdue tasks) for their team.	MUST
FR-TASK-10	Offline task-timer event queuing and synchronization are deferred to Release 1.1 (see Section 2.6.1). In Release 1.0, task timer start, pause, resume, and stop operations require an active network connection; a failed request shall be clearly reported as not recorded.	MUST

3.1.4 Timesheets (FR-TS)

ID	Requirement	Priority
FR-TS-01	The system shall automatically generate a draft timesheet for each Employee on a defined period boundary (default: weekly), aggregating attendance session totals and task-timer durations.	MUST
FR-TS-02	The system shall not allow an Employee to manually create a timesheet from scratch; manual entry is limited to reviewing and, where permitted, annotating the auto-generated draft.	MUST
FR-TS-03	The system shall allow an Employee to review a draft timesheet and submit it, transitioning its status from DRAFT to SUBMITTED (see Section 3.5.3).	MUST
FR-TS-04	The system shall allow a Manager to approve or reject a submitted timesheet; rejection shall require a rejection reason.	MUST
FR-TS-05	A rejected timesheet shall return to an editable state for employee correction and resubmission.	MUST
FR-TS-06	An approved timesheet shall remain an immutable approved snapshot and shall not be modified by the Employee. Source attendance or task-time changes shall not silently recalculate approved totals. A correction requires explicit Manager or Admin intervention, with the reason and actor recorded as part of the correction history.	MUST

ID	Requirement	Priority
FR-TS-07	Authorized users (Employee for their own records; Manager for their team; Admin organization-wide) shall be able to export approved timesheets to CSV, Excel, or PDF.	SHOULD

3.1.5 Dashboard, Reporting, and Engagement (FR-DASH)

ID	Requirement	Priority
FR-DASH-01	The system shall provide a Manager dashboard summarizing: active employees today, hours logged today, pending timesheet approvals, and task status counts.	MUST
FR-DASH-02	The system shall generate attendance reports (employee, date, clock-in, clock-out, total hours, sync status) filterable by date range and team.	MUST
FR-DASH-03	The system shall generate task reports (assigned, completed, pending, in-progress counts and completion rate) filterable by team and date range.	SHOULD
FR-DASH-04	The system shall display a named leaderboard showing the top three performers by task-completion percentage, with an organization-wide opt-out toggle for agencies that prefer not to use gamification.	SHOULD

3.1.6 Notifications (FR-NOTIFY)

ID	Requirement	Priority
FR-NOTIFY-01	The system shall integrate with the Telegram Bot API and allow a user to link their Telegram account to their Habeshan REMS profile.	MUST
FR-NOTIFY-02	The system shall send a Telegram notification to an Employee when their submitted timesheet is approved or rejected.	MUST
FR-NOTIFY-03	The system shall send a Telegram notification to a Manager when a timesheet is submitted for their approval.	MUST
FR-NOTIFY-04	The system shall send a Telegram reminder to an Employee who has not clocked in by a configurable organization-defined time.	SHOULD
FR-NOTIFY-05	All in-app notification events shall also be persisted to the `notifications` table, independent of Telegram delivery success.	MUST

3.1.7 Administration (FR-ADMIN)

ID	Requirement	Priority
FR-ADMIN-01	An Admin shall be able to create, view, update, and deactivate user accounts within their organization.	MUST
FR-ADMIN-02	An Admin shall be able to assign and change a user's role (Manager or	MUST

ID	Requirement	Priority
	Employee).	
FR-ADMIN-03	An Admin shall be able to create teams and assign a Manager and Employees to each team.	MUST
FR-ADMIN-04	An Admin shall be able to configure organization-level settings: display name, timezone, default currency (ETB), and notification preferences.	MUST
FR-ADMIN-05	An Admin shall have read access to all attendance, task, and timesheet data within their own organization only; cross-organization data access is prohibited at the API layer (see NFR-SEC-03).	MUST

3.2 External Interface Requirements

3.2.1 User Interfaces

The system shall provide a single responsive web application whose visible functions are scoped by role:

Role	Primary Screens / Functions
Employee	Clock-in/out widget, attendance history, task board (My Tasks), task timer, timesheet review, notifications, profile
Manager	Team dashboard, team attendance, task creation & board, timesheet approval queue, reports, notifications
Admin	User management, team management, organization settings, org-wide reports

Interface conventions: primary actions (Clock In, Submit Timesheet) shall be reachable in no more than two clicks from the relevant landing screen (see NFR-USE-01); status is communicated with both color and text, never color alone, for accessibility.

3.2.2 Hardware Interfaces

- Desktop and laptop computers with a standard keyboard/pointer input.
- Smartphones and tablets with a touchscreen, running a supported mobile browser.
- The system shall use the device's local clock and local storage only to support offline capture; it does not require camera, GPS, or biometric hardware in Release 1.0.

3.2.3 Software Interfaces

Interface	Technology / Version	Purpose
Database	PostgreSQL 15, accessed via GORM v2	Persistent storage for all organizational, attendance, task, and timesheet data
Backend runtime	Go 1.22+, Fiber v2 web framework	REST API host process
Offline store	IndexedDB, accessed via Dexie.js v4	Client-side queue for offline attendance

Interface	Technology / Version	Purpose
		and pending sync records
Notification channel	Telegram Bot API, via gopkg.in/telebot.v3	Outbound reminders and status alerts
Authentication	golang-jwt/jwt v5	JWT issuance and validation
Package management	npm (frontend), Go modules (backend)	Dependency management

3.2.4 Communication Interfaces

- All client-to-server communication shall use HTTPS in production; the API shall expose a JSON request/response body.
- The API shall return a consistent response envelope: `{ status: 'success' | 'fail', data: ..., message: ... }`.
- Telegram communication shall use the Telegram Bot HTTPS webhook interface.
- CORS shall be configured to permit requests only from the registered frontend origin(s).

3.3 Non-Functional Requirements

Per IEEE Std 830-1998, Clause 4.3.6, every non-functional requirement below is stated in measurable, verifiable terms; no requirement uses an unquantified adjective such as “fast”, “simple”, or “acceptable”.

3.3.1 Performance

ID	Requirement	Priority
NFR-PERF-01	The API shall respond to read operations (attendance, tasks, dashboard) within 300 ms at the 95th percentile under normal load.	MUST
NFR-PERF-02	The API shall respond to write operations (clock-in, task update) within 500 ms at the 95th percentile.	MUST
NFR-PERF-03	The manager dashboard shall render fully within 2.0 seconds on a simulated 4G connection.	MUST
NFR-PERF-04	Queued offline attendance records shall begin synchronizing within 5 seconds of connectivity being restored. This target does not apply to offline task-timer synchronization, which is deferred to Release 1.1.	MUST
NFR-PERF-05	The system shall support at least 200 concurrent active users per organization without degradation beyond the targets above.	SHOULD

3.3.2 Security

ID	Requirement	Priority
NFR-SEC-01	All protected API endpoints shall enforce RBAC middleware validated	MUST

ID	Requirement	Priority
	against the JWT claim set on every request.	
NFR-SEC-02	Local offline attendance data shall be encrypted at rest in IndexedDB using AES-256, with keys derived per-session and never persisted in plaintext.	MUST
NFR-SEC-03	Every data-access query shall be scoped by the requesting user's `organization_id` at the application layer, preventing cross-organization data leakage (horizontal privilege escalation).	MUST
NFR-SEC-04	The system shall use parameterized queries exclusively (via GORM) to prevent SQL injection.	MUST
NFR-SEC-05	JWT secrets and database credentials shall be stored only in server-side environment variables, never committed to source control.	MUST
NFR-SEC-06	Authentication endpoints shall be rate-limited to a maximum of 20 requests per 15 minutes per IP address.	MUST
NFR-SEC-07	CORS shall be restricted to the registered production frontend origin(s); wildcard origins are prohibited in production configuration.	MUST

3.3.3 Reliability

ID	Requirement	Priority
NFR-REL-01	An attendance record accepted locally (clock-in or clock-out) shall never be silently lost; every queued record shall either reach `SYNCED_VERIFIED` or `REJECTED_TAMPERED` and remain visible to the user until resolved.	MUST
NFR-REL-02	Duplicate synchronization of the same offline record shall be prevented via idempotent record UUIDs (see FR-ATT-08).	MUST
NFR-REL-03	Timesheet generation, attendance-total aggregation, and approval-state transitions shall each execute within a single database transaction with full rollback on failure.	MUST
NFR-REL-04	The system shall return a standardized error response, not an unhandled crash, on any downstream failure (database timeout, Telegram API failure).	MUST
NFR-REL-05	To prevent browser eviction of the offline IndexedDB queue under device storage pressure (especially on iOS Safari), the PWA client shall explicitly request persistent storage via the HTML5 StorageManager API (`navigator.storage.persist()`).	MUST
NFR-REL-06	The client interface shall provide a manual, user-initiated export of queued offline records at any time, allowing users to preserve pending data independent of automatic sync.	SHOULD

3.3.4 Availability

ID	Requirement	Priority
NFR-AVAIL-01	The backend API and database shall target 99.0% uptime measured monthly during the pilot phase.	SHOULD
NFR-AVAIL-02	Loss of backend connectivity shall not prevent local attendance capture; the offline queue shall remain fully functional independent of API availability.	MUST

3.3.5 Scalability

ID	Requirement	Priority
NFR-SCALE-01	The database schema shall support multiple isolated organizations without structural change (see `organizations` table, Section 3.4).	MUST
NFR-SCALE-02	The system shall support growth to at least 5,000 total employee accounts across all organizations on the MVP hosting tier before requiring infrastructure upgrade.	SHOULD

3.3.6 Maintainability

ID	Requirement	Priority
NFR-MAINT-01	The backend shall follow a layered architecture: handlers (HTTP), services (business logic/state machines), repositories (GORM data access), and middleware (auth, RBAC).	MUST
NFR-MAINT-02	Task and attendance state-transition rules shall be centralized in single service files, not duplicated across handlers.	MUST
NFR-MAINT-03	Environment-specific configuration shall be externalized to `.env` files and never hardcoded.	MUST
NFR-MAINT-04	The codebase shall use version control (Git) with all changes reviewed via pull request before merge to the main branch.	MUST

3.3.7 Usability

ID	Requirement	Priority
NFR-USE-01	The primary Employee workflows (clock in, clock out, update task status) shall be reachable in no more than 2 clicks from the app's main screen.	MUST
NFR-USE-02	All asynchronous actions shall display a loading indicator and disable the triggering control while the request is in progress, preventing duplicate submission.	MUST
NFR-USE-03	Task and timesheet status shall be represented with a distinct color code in addition to text, so status is legible without reading (e.g., grey = To Do, blue	MUST

ID	Requirement	Priority
	= In Progress, green = Completed).	
NFR-USE-04	Error messages returned to the UI shall be human-readable; raw stack traces or internal error codes shall never be exposed to end users.	MUST

3.3.8 Portability

ID	Requirement	Priority
NFR-PORT-01	The web client shall function correctly on the latest two major versions of Chrome, Firefox, Edge, and Safari on Windows, macOS, Linux, Android, and iOS.	MUST
NFR-PORT-02	The backend shall run in a standard Linux container with no host-specific dependencies, enabling migration between hosting providers without code change.	SHOULD

3.4 Database Requirements

The Habeshan REMS relational schema comprises eleven tables, hosted on PostgreSQL 15 and accessed exclusively through the GORM object-relational mapper using parameterized queries.

Every table uses a UUID primary key (`id`, default `gen_random_uuid()`) and `timestamptz` timestamp columns, per Constraint C8 (Section 2.5). Two schema-wide conventions govern which tables carry which additional columns:

- **Soft delete (`deleted_at`):** present only on `organizations` and `users`. No other table carries a `deleted_at` column — attendance, task, timesheet, and notification records are never soft-deleted; they are immutable history once written.
- **Tenant scoping (`org_id`):** present on `users`, `teams`, `attendance_logs`, `tasks`, `task_time_logs`, `timesheets`, and `notifications` — the tables that are queried directly by the API and therefore need a direct `WHERE org_id = ?` clause for NFR-SEC-03. It is deliberately absent from `team_members` and `task_assignments` (junction tables, always accessed through a parent table that already carries `org_id`) and from `telegram_subscribers` (a 1:1 extension of `users`, which already carries `org_id`). Adding `org_id` to these three tables would be redundant and would create a second source of truth for an organization boundary that is already enforced upstream.

Multi-tenancy is achieved by applying organization scoping to every organization-owned data access. Tables with a direct `org_id` column are filtered by the authenticated organization. Junction and extension tables are accessed through joins to their organization-scoped parent or owning records. The application shall enforce this boundary on every relevant query (NFR-SEC-03).

Table 1 — organizations

The tenant / billing root. One row per subscribing agency.

Column	Type	Constraints	Description
id	uuid	PK, DEFAULT gen_random_uuid()	Unique organization identifier
name	VARCHAR(150)	NOT NULL	Organization display name
currency	VARCHAR(3)	DEFAULT 'ETB'	Billing currency
timezone	VARCHAR(50)	DEFAULT 'Africa/Addis_Ababa'	Default org timezone
seat_count	INT	DEFAULT 0	Purchased/allocated employee seats (Release 1.1 billing)
subscription_status	ENUM('trial','active','suspended')	DEFAULT 'trial'	Billing state (Release 1.1)
created_at	timestamptz	DEFAULT CURRENT_TIMESTAMP	Organization creation time
updated_at	timestamptz	DEFAULT CURRENT_TIMESTAMP	Last modification time
deleted_at	timestamptz	NULL — soft delete	Soft-delete marker

Table 2 — users

All Admin, Manager, and Employee accounts across all organizations.

Column	Type	Constraints	Description
id	uuid	PK, DEFAULT gen_random_uuid()	Unique user identifier
org_id	uuid	FK → organizations(id), NOT NULL	Owning organization
email	VARCHAR(255)	NOT NULL — unique per organization, see composite constraint below	Login credential
password_hash	VARCHAR(255)	NOT NULL	bcrypt hash, cost ≥ 10
full_name	VARCHAR(100)	NOT NULL	Display name
phone	VARCHAR(20)	NULL	Contact number
role	ENUM('admin','manager','emplo	NOT NULL	RBAC role

Column	Type	Constraints	Description
	yee')		
status	ENUM('active','inactive')	DEFAULT 'active'	Account status
created_at	timestamptz	DEFAULT CURRENT_TIMESTAMP	Account creation time
updated_at	timestamptz	DEFAULT CURRENT_TIMESTAMP	Last modification time
deleted_at	timestamptz	NULL — soft delete	Soft-delete marker

Composite constraint: `UNIQUE (org_id, email) WHERE deleted_at IS NULL` — an email address is unique within a single organization among active accounts only. The same email may be used to register under a different organization, and becomes available for reuse once the original account is soft-deleted.

Table 3 — teams

Groups employees under a single Manager. (Admins have global override without needing assignment here.)

Column	Type	Constraints	Description
id	uuid	PK, DEFAULT gen_random_uuid()	Unique team identifier
org_id	uuid	FK → organizations(id), NOT NULL	Owning organization
manager_id	uuid	FK → users(id), NOT NULL	Team's supervising manager
name	VARCHAR(100)	NOT NULL	Display name
created_at	timestamptz	DEFAULT CURRENT_TIMESTAMP	Team creation time
updated_at	timestamptz	DEFAULT CURRENT_TIMESTAMP	Last modification time

No `deleted_at` on this table — soft delete is scoped to `organizations` and `users` only.

Table 4 — team_members

Many-to-many mapping of employees to teams. No `org_id`: this junction table is always accessed by joining through `teams` or `users`, both of which already carry `org_id`.

Column	Type	Constraints	Description
team_id	uuid	PK, FK → teams(id)	Team side of the junction
user_id	uuid	PK, FK → users(id)	Employee side of the junction

Table 5 — attendance_logs

One row per clock-in/clock-out session; the record synchronized from the offline queue.

Column	Type	Constraints	Description
id	uuid	PK, DEFAULT gen_random_uuid()	Unique attendance record identifier
org_id	uuid	FK → organizations(id), NOT NULL	Owning organization
user_id	uuid	FK → users(id), NOT NULL	Employee who clocked in
clocked_in_at	timestamptz	NOT NULL	Session start time
clocked_out_at	timestamptz	NULL	Session end time, null while active
total_hours	DECIMAL(6,2)	NULL, computed on clock-out	Session duration in hours
sync_status	ENUM('offline_logged','pending_sync','synced_verified','rejected_tampered')	DEFAULT 'pending_sync'	Offline-sync state (Section 3.5.1)
device_hash	VARCHAR(128)	NOT NULL	Client-computed tamper-evidence hash
record_uuid	uuid	UNIQUE, NOT NULL	Client-generated ID for idempotent sync
created_at	timestamptz	DEFAULT CURRENT_TIMESTAMP	Server insert time
updated_at	timestamptz	DEFAULT CURRENT_TIMESTAMP	Last modification time

No `deleted_at` on this table — attendance history is immutable once written; corrections happen via the flagged-review and Admin-correction workflow (FR-ATT-07), not deletion.

Table 6 — tasks

Units of work created by a Manager or Admin.

Column	Type	Constraints	Description
id	uuid	PK, DEFAULT gen_random_uuid()	Unique task identifier
org_id	uuid	FK → organizations(id), NOT NULL	Owning organization
created_by	uuid	FK → users(id), NOT NULL	Manager/Admin who created the task
title	VARCHAR(150)	NOT NULL	Task title

Column	Type	Constraints	Description
description	TEXT	NULL	Task detail
priority	ENUM('low','medium','high')	DEFAULT 'medium'	Task priority
status	ENUM('to_do','in_progress','paused','blocked','completed')	DEFAULT 'to_do'	Kanban status (Section 3.5.2)
due_date	DATE	NULL	Target completion date
created_at	timestamptz	DEFAULT CURRENT_TIMESTAMP	Task creation time
updated_at	timestamptz	DEFAULT CURRENT_TIMESTAMP	Last modification time

Table 7 — task_assignments

Many-to-many mapping supporting multi-employee task assignment. No `org_id`, for the same reason as `team_members`.

Column	Type	Constraints	Description
task_id	uuid	PK, FK → tasks(id)	Task side of the junction
user_id	uuid	PK, FK → users(id)	Assigned employee
assigned_at	timestamptz	DEFAULT CURRENT_TIMESTAMP	Assignment time

Table 8 — task_time_logs

Individual timer start/pause/resume/stop events, the basis for auto-generated timesheets.

Column	Type	Constraints	Description
id	uuid	PK, DEFAULT gen_random_uuid()	Unique time-log identifier
org_id	uuid	FK → organizations(id), NOT NULL	Owning organization
task_id	uuid	FK → tasks(id), NOT NULL	Associated task
user_id	uuid	FK → users(id), NOT NULL	Employee who logged the time
started_at	timestamptz	NOT NULL	Timer start

Column	Type	Constraints	Description
stopped_at	timestamptz	NULL	Timer stop, null while running
duration_minutes	INT	NULL, computed on stop	Segment duration
pause_reason	VARCHAR(50)	NULL	Reason code if this segment ended in a pause
sync_status	ENUM('offline_logged', 'pending_sync','synced_verified','rejected_tampered')	DEFAULT 'pending_sync'	Offline-sync state (reserved for Release 1.1 offline task-timer sync, FR-TASK-10)
device_hash	VARCHAR(128)	NOT NULL	Client-computed tamper-evidence hash
record_uuid	uuid	UNIQUE, NOT NULL	Client-generated ID for idempotent sync
created_at	timestamptz	DEFAULT CURRENT_TIMESTAMP	Server insert time
updated_at	timestamptz	DEFAULT CURRENT_TIMESTAMP	Last modification time

Table 9 — timesheets

System-generated, period-based rollups submitted for manager approval.

Column	Type	Constraints	Description
id	uuid	PK, DEFAULT gen_random_uuid()	Unique timesheet identifier
org_id	uuid	FK → organizations(id), NOT NULL	Owning organization
user_id	uuid	FK → users(id), NOT NULL	Owning employee
reviewed_by	uuid	FK → users(id), NULL	Manager who approved/rejected the timesheet
period_start	DATE	NOT NULL	Period start date
period_end	DATE	NOT NULL	Period end date

Column	Type	Constraints	Description
total_hours	DECIMAL(6,2)	NOT NULL, auto-computed	Aggregated hours for the period
status	ENUM('draft','submitted','approved','rejected')	DEFAULT 'draft'	Approval workflow state (Section 3.5.3)
rejection_reason	TEXT	NULL	Required when status = rejected
created_at	timestamptz	DEFAULT CURRENT_TIMESTAMP	Generation time
updated_at	timestamptz	DEFAULT CURRENT_TIMESTAMP	Last modification time

Table 10 — notifications

In-app notification log, persisted independent of Telegram delivery.

Column	Type	Constraints	Description
id	uuid	PK, DEFAULT gen_random_uuid()	Unique notification identifier
org_id	uuid	FK → organizations(id), NOT NULL	Owning organization
user_id	uuid	FK → users(id), NOT NULL	Recipient
type	VARCHAR(50)	NOT NULL	e.g., timesheet_approved, timesheet_rejected, clock_in_reminder
message	text	NOT NULL	Notification body
read	boolean	DEFAULT false	Read status
created_at	timestamptz	DEFAULT CURRENT_TIMESTAMP	Notification creation time

No `updated_at` or `deleted_at` on this table — notifications are write-once records.

Table 11 — telegram_subscribers

Links a user account to a Telegram chat for outbound notifications. No `org_id`: this table is a 1:1 extension of `users` and inherits organization scope through `user_id`.

Column	Type	Constraints	Description
user_id	uuid	PK, FK → users(id)	Linked user account
chat_id	varchar(50)	NOT NULL	Telegram chat identifier

Column	Type	Constraints	Description
is_active	boolean	DEFAULT true	Whether notifications are currently enabled
linked_at	timestamptz	DEFAULT CURRENT_TIMESTAMP	Time the Telegram link was created

3.4.1 Entity-Relationship Diagram

The eleven tables above relate as follows: `organizations` is the tenant root referenced directly by `users`, `teams`, `attendance_logs`, `tasks`, `task_time_logs`, `timesheets`, and `notifications`. `users` is referenced by `teams.manager_id`, by both sides of `team_members`, by `tasks.created_by`, by both sides of `task_assignments`, by `attendance_logs.user_id`, `task_time_logs.user_id`, `timesheets.user_id` and `reviewed_by`, `notifications.user_id`, and by `telegram_subscribers.user_id`. `tasks` is referenced by `task_assignments.task_id` and `task_time_logs.task_id`.

3.4.2 Data Integrity Rules

- Every user, task, and attendance record shall reference a valid, existing `organization_id` (directly or transitively via `user_id`).
- A user's email address shall be unique only within the scope of their organization (`org_id`), not globally across the system; soft-deleted accounts (`deleted_at IS NOT NULL`) are excluded from this uniqueness check, allowing the email to be reused for a new registration in the same or a different organization.
- An attendance record shall belong to a valid, active employee at the time of capture.
- A `task_assignments` row shall reference a user who belongs to the same organization as the task.
- A `teams.manager_id` shall reference an active Manager account in the same organization as the team.
- Every `team_members` row shall connect a team and user belonging to the same organization.
- Duplicate active attendance sessions (a second open clock-in while one is already open) shall be rejected (see BR-01).
- Required fields listed as NOT NULL shall be validated at both the API and database layers (defense in depth).

3.4.3 Backup and Recovery

The system shall rely on Aiven Cloud's managed automated daily backups of the PostgreSQL instance, with point-in-time recovery retained for a minimum of 7 days during the pilot phase. A full schema migration history shall be maintained in version control to allow environment reconstruction independent of the managed backup service.

3.5 State Machines

Three state machines govern the core lifecycles of the system. In each case, only the transitions listed are valid; the backend shall reject any other transition with HTTP 400 and a descriptive error message, and shall record every transition with an actor and timestamp.

3.5.1 Attendance Synchronization State Machine

From State	To State	Trigger	Actor
(new)	OFFLINE_LOGGED	Clock action captured with no connectivity	Employee device
(new)	PENDING_SYNC	Clock action captured with connectivity present	Employee device
OFFLINE_LOGGED	PENDING_SYNC	Connectivity restored (automatic)	System
PENDING_SYNC	SYNCED_VERIFIED	Device hash and timestamp validate successfully	System
PENDING_SYNC	REJECTED_TAMPERED	Device hash mismatch or implausible local-clock drift detected	System

3.5.2 Task Status State Machine

From State	To State	Trigger	Actor
To Do	In Progress	Employee starts the task timer	Employee
In Progress	Paused	Employee pauses the timer and selects a reason	Employee
Paused	In Progress	Employee resumes the timer	Employee
In Progress	Blocked	Employee marks the task blocked on a dependency	Employee
Blocked	In Progress	Blocking dependency resolved	Employee
In Progress	Completed	Employee marks the task complete	Employee
Paused	Completed	Employee marks complete directly from a paused state (flagged for Manager review)	Employee

3.5.3 Timesheet Approval State Machine

From State	To State	Trigger	Actor
(new)	Draft	System auto-generates the timesheet at the period boundary	System (scheduled job)
Draft	Submitted	Employee reviews and submits the timesheet	Employee
Submitted	Approved	Manager approves the timesheet	Manager
Submitted	Rejected	Manager rejects the timesheet with a reason	Manager
Rejected	Submitted	Employee corrects and resubmits	Employee

3.6 Business Rules

ID	Business Rule
BR-01	An employee shall not have more than one open (unclosed) attendance session at a time.
BR-02	An employee shall clock in before they are permitted to clock out; a clock-out without a matching open clock-in is rejected.
BR-03	Only a Manager assigned to a team, or an Admin, may approve or reject that team's timesheets.
BR-04	A rejected timesheet may be corrected and resubmitted by the owning employee; it may not be resubmitted by any other user.
BR-05	Only a Manager or Admin may create or assign tasks; an Employee may update the status of tasks assigned to them but not create new tasks.
BR-06	A user may only perform operations permitted by their assigned role, enforced identically at the API layer regardless of what the UI displays.
BR-07	An approved timesheet shall remain an immutable approved snapshot. Any post-approval correction requires explicit Manager or Admin action, with the reason and actor recorded as an override; source-data changes shall not silently rewrite the approved totals.
BR-08	Every offline attendance record must reach either SYNCED_VERIFIED or REJECTED_TAMPERED; no record may remain indefinitely unresolved once connectivity is available.
BR-09	An employee may view only their own attendance and timesheet records unless they hold a Manager or Admin role for the relevant team/organization.
BR-10	A Manager may access only the employees and teams explicitly assigned to them. A request-supplied `team_id` or `user_id` shall never substitute for an authorization check against the Manager's actual team assignment. Only an Admin has organization-wide access.
BR-11	Pausing a task timer requires a reason; pause reasons are inherently private and visible only to the employee and their Manager, never to peers.
BR-12	A task may be assigned to multiple employees; each assigned employee accrues their own independent `task_time_logs` entries.
BR-13	All monetary and billing fields default to ETB; no other currency is supported in Release 1.0.
BR-14	Data belonging to one organization is never returned in a response to a user authenticated under a different organization, regardless of role.
BR-15	A user's email is unique per organization, not system-wide; the same individual may hold separate accounts under different organizations using the same email, and a soft-deleted account's email becomes available for reuse.

4. System Models

This section presents the visual models that complement the textual requirements of Section 3, in conformance with IEEE Std 830-1998 guidance (Clause 5.3.7) that complex logic be illustrated graphically as well as described in prose.

4.1 Use Case Diagram

Three actors — Admin, Manager, Employee — interact with the system's use cases as summarized in the role table of Section 2.3 and detailed in the use case catalog, Appendix A.

4.2 Offline Attendance Activity Flow

The following describes the end-to-end activity flow for an employee clock-in event, which underlies the state machine in Section 3.5.1:

1. Employee opens the app and taps Clock In.
2. Client checks network connectivity. 3a. If online: the request is sent directly to `POST /api/v1/attendance/clock-in`; the server validates, stores the record as PENDING_SYNC, and immediately verifies it. 3b. If offline: the record is written to the local Dexie.js/IndexedDB queue with status OFFLINE_LOGGED and an AES-256-encrypted payload including the device hash.
3. The Service Worker listens for a connectivity-restored browser event.
4. On reconnection, the client automatically POSTs all queued records to `/api/v1/attendance/sync` in a single batch.
5. The server verifies each record's device hash and timestamp plausibility, marking each SYNCED_VERIFIED or REJECTED_TAMPERED.
6. The client removes successfully synced records from the local queue and surfaces any REJECTED_TAMPERED record to the employee with guidance to contact their Manager.

4.3 Key Interaction Sequences

4.3.1 Clock-In Sequence (Online) Employee → React Client → Go API → PostgreSQL → Go API → React Client → Employee. The API validates the JWT, inserts the `attendance_logs` row, and returns the confirmed record with server-assigned id.

4.3.2 Automated Timesheet Generation Sequence Scheduled Job → Go API (timesheet service) → PostgreSQL (reads `attendance_logs` and `task_time_logs` for the period) → PostgreSQL (writes a new `timesheets` row, status = Draft) → Notification Service → Telegram → Employee. The employee is notified that a new timesheet is ready for review.

4.3.3 Timesheet Approval Sequence Manager → React Client → Go API (`PUT /api/v1/timesheets/:id/approve`) → PostgreSQL (status update within a transaction, sets `reviewed_by`) → Notification Service → Telegram → Employee.

4.4 Entity-Relationship Diagram

The complete entity-relationship structure for the 11-table schema is presented in Section 3.4.1, alongside the full column-level table definitions in Section 3.4.

---

5. Appendices

Appendix A — Use Case Catalog

UC#	Name	Actor	Precondition	Main Flow (Summary)	FR Reference
UC-01	Register / Login	Any user	User has a valid invite or account	User submits credentials → system validates → issues JWT	FR-AUTH-01–04
UC-02	Reset Password	Any user	User has a registered email	User requests reset → receives token → sets new password	FR-AUTH-07
UC-03	Clock In / Out (Online)	Employee	User authenticated	Employee taps Clock In/Out → request sent directly → server verifies and stores	FR-ATT-01, 02
UC-04	Clock In / Out (Offline)	Employee	No connectivity	Action queued locally → auto-synced on reconnect → server verifies hash	FR-ATT-03–08
UC-05	Create & Assign Task	Manager	User authenticated as Manager/Admin	Manager fills task form → selects one or more employees → system creates task and assignments	FR-TASK-01, 02
UC-06	Work on Task (Timer)	Employee	Task assigned to user	Employee starts timer → optionally pauses with reason → stops → system logs segments	FR-TASK-05–08
UC-07	Auto-Generate Timesheet	System	Period boundary reached	Scheduled job aggregates attendance + task-timer data → creates Draft timesheet → notifies employee	FR-TS-01
UC-08	Submit Timesheet	Employee	Draft timesheet exists	Employee reviews → submits → status becomes Submitted	FR-TS-02, 03
UC-09	Approve / Reject Timesheet	Manager	Timesheet is Submitted	Manager reviews → approves or rejects with reason → employee notified	FR-TS-04–06
UC-10	View Manager	Manager	User authenticated	Manager opens dashboard → system computes and displays	FR-DASH-

UC#	Name	Actor	Precondition	Main Flow (Summary)	FR Reference
	Dashboard		as Manager	live team metrics	01
UC-11	Export Report	Manager / Admin	Data exists for the selected range	User selects range and format → system generates CSV/Excel/PDF	FR-DASH-02, 03, FR-TS-07
UC-12	Manage Users & Teams	Admin	User authenticated as Admin	Admin creates/edits users, assigns roles, builds teams	FR-ADMIN-01–03
UC-13	Receive Telegram Notification	Employee / Manager	Telegram account linked	Triggering event occurs → bot sends message → notification also persisted in-app	FR-NOTIFY-01–05

Appendix B — REST API Surface

B.1 Base Configuration

Property	Value
Base URL (development)	http://localhost:8080/api/v1
Base URL (production)	https://api.habeshanrems.com/api/v1 (placeholder — assigned at deployment)
Data format	JSON
Authentication	Bearer token (JWT) in the Authorization header
Response envelope	`{ status: 'success' | 'fail', data: ..., message: ... }`

B.2 Authentication

Method	Path	Body / Notes	Access
POST	/auth/register	{ email, password, full_name, phone, org_id }	Public
POST	/auth/lookup	{ email } — resolves organization context; does not authenticate	Public
POST	/auth/login	{ email, password, org_id } — authenticates within the organization selected by lookup	Public
POST	/auth/forgot-password	{ email }	Public
POST	/auth/reset-	{ token, new_password }	Public

Method	Path	Body / Notes	Access
	password		
GET	/auth/me	Returns the current authenticated user's profile	Authenticated

B.3 Attendance

Method	Path	Body / Notes	Access
POST	/attendance/clock-in	{ device_hash, record_uuid, timestamp }	Employee
POST	/attendance/clock-out	{ device_hash, record_uuid, timestamp }	Employee
POST	/attendance/sync	Bulk array of queued offline records	Employee
GET	/attendance/me	Paginated; own history	Employee
GET	/attendance/team	Paginated; filtered by team	Manager
GET	/attendance/org	Paginated; org-wide	Admin

B.4 Tasks

Method	Path	Body / Notes	Access
POST	/tasks	{ title, description, priority, due_date, assigned_user_ids[] }	Manager / Admin
GET	/tasks/my-tasks	Tasks assigned to the requesting employee	Employee
GET	/tasks/team	Paginated; all tasks for the manager's team	Manager
PATCH	/tasks/:id/status	{ status }	Employee (assigned) / Manager
POST	/tasks/:id/timer/start	—	Employee (assigned)
POST	/tasks/:id/timer/pause	{ reason }	Employee (assigned)
POST	/tasks/:id/timer/resume	—	Employee (assigned)
POST	/tasks/:id/timer/stop	—	Employee (assigned; online only in Release 1.0)

No task-timer bulk-sync endpoint is defined in Release 1.0; offline synchronization is FR-TASK-10, deferred to Release 1.1. The `:id` path parameters and the `assigned_user_ids[]` request field are API-contract names and are not required to match the underlying database column names in Section 3.4 (`tasks.id`, `task_assignments.user_id`); the REST layer's naming is a separate, stable contract from the schema.

B.5 Timesheets

Method	Path	Body / Notes	Access
GET	/timesheets/me	Own timesheets, any status	Employee
PUT	/timesheets/:id/submit	—	Employee (owner)
GET	/timesheets/pending	Submitted timesheets awaiting this manager's approval	Manager
PUT	/timesheets/:id/approve	—	Manager
PUT	/timesheets/:id/reject	{ rejection_reason }	Manager
GET	/timesheets/:id/export	?format=csv|xlsx|pdf	Employee (own) / Manager (team) / Admin (org)

B.6 Dashboard, Reports, Admin, and Notifications

Method	Path	Body / Notes	Access
GET	/dashboard/manager	Live team summary metrics	Manager
GET	/dashboard/leaderboard	Top 3 performers (opt-out per organization)	Employee / Manager
GET	/reports/attendance	?from=&to=&team_id=	Manager / Admin
GET	/reports/tasks	?from=&to=&team_id=	Manager / Admin
POST	/admin/users	{ email, full_name, role, team_id } — role may be Manager or Employee only; Admin creation through this endpoint is prohibited (FR-ADMIN-02)	Admin
PATCH	/admin/users/:id	{ role, status }	Admin
POST	/admin/teams	{ name, manager_id }	Admin
GET	/notifications	Paginated; own notifications	Authenticated
POST	/notifications/telegram/link	{ telegram_chat_id }	Authenticated
POST	/telegram/webhook	Telegram-originated payload	System (bot)

B.7 Standard Error Response Codes

HTTP Code	Meaning	Typical Cause
200	OK	Successful read operation
201	Created	Successful write operation (task, timesheet, etc.)
400	Bad Request	Invalid input, failed validation, illegal state transition
401	Unauthorized	Missing, expired, or invalid JWT
403	Forbidden	Valid token but insufficient role or resource ownership
404	Not Found	Resource does not exist or is not accessible to the caller
409	Conflict	Duplicate resource (e.g., duplicate open attendance session)
500	Internal Server Error	Unexpected backend failure

Appendix C — Functional Requirements Traceability Matrix

Use Case	Functional Requirement(s)	API Endpoint(s)	Frontend Screen	Verification Method
UC-03/04 Clock In/Out	FR-ATT-01–08	POST /attendance/clock-in, /clock-out, /sync	/dashboard (clock widget)	Integration test; airplane-mode offline simulation
UC-05 Create & Assign Task	FR-TASK-01, 02	POST /tasks	/tasks (manager view)	Unit + integration test on multi-assignee logic
UC-06 Work on Task (Timer)	FR-TASK-05–08	POST /tasks/:id/timer/*	/tasks/:id	Unit test on timer state transitions
UC-07 Auto-Generate Timesheet	FR-TS-01	(scheduled job, no direct endpoint)	N/A	Scheduled-job test; verify Draft row created with correct totals
UC-08 Submit Timesheet	FR-TS-02, 03	PUT /timesheets/:id/submit	/timesheets	Role-based E2E test
UC-09 Approve/Reject Timesheet	FR-TS-04–06	PUT /timesheets/:id/approve, /reject	/manager/timesheets	Role-based E2E test
UC-10 Manager	FR-DASH-01	GET /dashboard/mana	/manager/dashbo	Load-time performance test

Use Case	Functional Requirement(s)	API Endpoint(s)	Frontend Screen	Verification Method
Dashboard		ger	ard	(NFR-PERF-03)
UC-11 Export Report	FR-DASH-02, 03, FR-TS-07	GET /reports/*, /timesheets/:id/export	/reports	Output-format validation test
UC-12 Manage Users & Teams	FR-ADMIN-01–05	POST/PATCH /admin/users, /admin/teams	/admin	RBAC boundary test (cross-org isolation, NFR-SEC-03)
UC-13 Telegram Notification	FR-NOTIFY-01–05	POST /notifications/telegram/link, /telegram/webhook	/settings/notifications	Webhook integration test

Appendix D — Environment Variables Template

The following environment variables shall be defined in a `.env` file in the backend root directory and never committed to version control.

Variable	Example Value	Purpose
PORT	8080	Backend server port
APP_ENV	development | production	Environment mode
DB_HOST	pg-host.aivencloud.com	Database host (Aiven)
DB_PORT	5432	PostgreSQL port
DB_USER	avnadmin	Database username
DB_PASSWORD	your_secure_password	Database password
DB_NAME	habeshan_rems	Database name
JWT_SECRET	minimum_32_char_random_string	JWT signing secret
JWT_EXPIRES_IN	24h	Token expiry duration
FRONTEND_URL	https://app.habeshanrems.com	CORS allowed origin
TELEGRAM_BOT_TOKEN	123456:ABC-DEF...	Telegram Bot API token
RATE_LIMIT_WINDOW	900000	Rate-limit window (ms) — 15

Variable	Example Value	Purpose
		minutes
RATE_LIMIT_MAX	20	Max auth requests per window per IP

Appendix E — Roadmap Summary (Non-Normative)

Reproduced from Section 2.6 for quick reference; these items are explicitly not part of the Release 1.0 requirements in Section 3.

- **Release 1.1:** Chapa / Telebirr subscription billing, per-seat plan management, invoicing, and offline task-timer synchronization (FR-TASK-10).
- **Release 2.0+:** Automated payroll disbursement, client portal, Git/IDE proof-of-work integration, native mobile apps, AI productivity analytics, hardened multi-organization self-serve growth.